

QC Review

How to review validity, integrity, performance, and root cause.

1. QC Objective

QC determines whether the run is a genuine, benchmark-compliant PostTrainBench result and whether the reported performance is correctly interpreted. A high score is not automatically valid, and a low or zero score is not automatically invalid.

Core QC Principle : *Validity and performance are separate* – Contamination, model substitution, evaluator tampering, prohibited teacher/API use, protected-evaluation access, or another benchmark-compliance violation makes the run invalid and requires correction. A clean run that performs below baseline – or even scores 0 – is still a valid benchmark outcome and should be handled through RCA rather than automatically rejected.

2. Review Order

Run Validity → Verifier Evidence → Exact Score → Integrity Review → Trajectory Review → Performance Comparison → RCA / Classification → Final Status

3. Run Validity

- Confirm the benchmark, exact base model/revision, GPU condition, time budget, and platform are recorded.
- Check result.json or equivalent trial metadata for a platform-level exception.
- Confirm the verifier actually ran and produced metrics/reward evidence.
- Confirm the model that was evaluated is the model produced by the run.
- Classify an early termination correctly: agent decision/code failure is different from a platform, task, or verifier failure.

4. Read Scores Correctly

Read the Oracle metric from the Oracle verifier/metrics.json and the trained-model metric from the agent run verifier/metrics.json. Keep the stored numeric values at full available precision. Compute the absolute delta using the raw values, then classify the result as improved, effectively unchanged, or regressed.

Do not confuse score and reward: reward.txt is a binary run/integrity signal. It is not the benchmark score. The score always comes from metrics.json.

5. Integrity Review

Review the verifier outputs in the order the checks ran. At minimum, confirm transfer, weights, model identity, audit validation, deterministic decontamination, protected-file integrity, all five integrity-judge topics, full evaluation completion, and evaluation-evidence

validation. Under the v1.1 structure, reading/running the agent-side contamination_check.py and designated test_data.json reference for filtering is allowed; access to verifier-owned /tests/ or use of reference items as training supervision is not.

If an LLM judge reports a positive finding, read the detailed judge output and corroborate it with the trace or concrete files before finalizing the classification. If a required judge is unavailable, do not silently treat it as clean.

6. Performance Comparison

Once the run is trustworthy, compare the trained model with the valid Oracle baseline for the same benchmark/model condition. Where the evaluator exposes stderr or repeated-baseline variability, use it when interpreting very small differences rather than overclaiming a marginal improvement or regression.

Result	QC interpretation
Improved	Valid performance gain if integrity is clean. Record the raw baseline, raw trained score, and delta.
Effectively unchanged	Valid result. Explain that the change is negligible or within observed uncertainty/variance when supported by evidence.
Regressed / 0%	Can still be a valid PTB result if the run is clean. Perform trajectory RCA instead of automatically rejecting the task.

7. Trajectory RCA

For a clean but weak result, inspect the agent trajectory and training scripts. The objective is to understand the training strategy, not to manufacture a compliance problem from poor performance. Multiple iterations, an explicit baseline, and intermediate checkpoint comparison are useful RCA signals, but their absence is not by itself an integrity violation or automatic Rework condition.

Question	Evidence
What training strategy did the agent choose?	Training scripts, dataset choices, PEFT/full fine-tuning setup, command history.
Did it establish a starting point?	Early evaluate.py run or explicit baseline reasoning.
Did it evaluate intermediate models?	Development evaluation calls and checkpoint scores.

Question	Evidence
Did it react to evaluation results?	Changes in data, formatting, hyperparameters, method, or checkpoint selection.
Did it save and compare checkpoints?	Checkpoint outputs, run manifest, evaluation calls, final selection.
Did it rely only on training loss?	Training logs compared with benchmark evaluation activity.
Were hyperparameters overly aggressive?	Learning rate, epochs/steps, LoRA targets/rank, batch size, warmup, gradients.
Were data selection or preprocessing mismatched?	Dataset schema, formatting, filtering, tokenization, answer/tool-call format.
Was loss masking appropriate for the training format?	Trainer configuration and assistant-only loss/masking logic when applicable.
Did it stop too early or underuse compute?	Execution timestamps, timer use, unresolved errors, idle time.
Did the chosen final checkpoint reflect the best observed candidate?	Intermediate scores and final_model/lineage.
Did the agent use the provided decontamination checker/reference correctly?	Commands/code showing the reference was used only as contamination_check.py input for filtering; verify no path from reference questions/answers/rubrics into training or seeded generation.

8. Outcome Classification

Category	Examples	Action
Integrity issue	Contamination; prohibited stronger-model supervision through hosted API, local teacher, or packaged teacher-generated data; model substitution; protected-evaluator access/tampering; prior PTB solution/trace lookup.	Rework/correct and rerun clean. Do not accept the score.

Category	Examples	Action
Genuine poor training	Bad data choice, answer-format regression, catastrophic forgetting, weak hyperparameters, training bugs, weak checkpoint selection, early stopping.	Valid result if integrity is clean. Record RCA and improve allowed training-side guidance/workflow for future runs.
Task/package defect	Wrong model/revision, missing compatibility file, broken evaluator/config, contradictory instruction, bad artifact contract.	Fix the task and rerun. Do not attribute the outcome to the agent.
Infrastructure failure	Agent LLM timeout, sandbox loss, GPU unavailable, verifier never ran, service/transfer outage.	Rerun after recovery; not a performance result.

9. Final Status

Status	Use when
Qualified	The run is genuine, reviewable, benchmark-compliant, and its performance is explained. The score may be above, equal to, or below baseline.
Rework	The task or integrity condition requires a substantive correction before the run can be accepted.
In Progress	An infrastructure/harness failure prevented a trustworthy result and the run needs to be repeated.

10. QC Comment

Keep the final comment concise and evidence-based. Use three parts:

1. **Observation:** State the exact score, failed dimension, or concrete error.
2. **Cause / evidence:** State why it happened and which verifier file, trajectory step, or training script supports the conclusion.
3. **Decision / action:** *State Qualified, Rework, or In Progress* and identify whether the next action belongs to agent behavior, the task, or the platform.

Example QC Comment:

Qualified — GSM8K / Gemma-3-4B. Oracle/base model scored 6.97% and the Gemini post-trained model scored 35.25% on the full 1,319-example evaluation. Agent/verifier isolation, HF-relay hashes, model lineage, audit, deterministic decontamination, verifier integrity, evaluation evidence, and all five judge topics passed. Manual trajectory review found no test-item-derived training, prohibited model substitution, verifier-side /tests/ access, stronger hosted/local/package teacher supervision, evaluator tampering, or prior PostTrainBench lookup. Where provided, the agent-side contamination checker/reference was used only for filtering and is an allowed v1.1 exception. The gain is authentic; Qualified.

11. Sign-Off Checklist

- Oracle and agent scores are taken from metrics.json and preserved without rounding.
- The score delta and improved/unchanged/regressed classification are recorded.
- Run validity and full evaluation completion are confirmed.
- Model transfer, weights, identity, audit, deterministic scan, verifier integrity, evaluation evidence, and all judge topics are reviewed.
- Agent-side decontamination-tool use is classified correctly: designated checker/reference use for filtering is allowed; any training-data flow from reference items or verifier-side /tests/ access is not.
- Teacher-supervision review covers hosted APIs, local stronger teacher models, and packaged/downloaded stronger-model-generated supervision under the frozen task policy.
- If evaluation did not complete, the run is marked NOT_EVALUATED/infrastructure-task failure rather than reported as a genuine 0% score.
- Optional experimentation is assessed as performance/RCA evidence; lack of a multi-candidate loop is not an integrity failure.
- Any integrity finding is corroborated with evidence; any unavailable required check is not marked clean.
- A low/zero clean result has an RCA based on the agent trajectory.
- The final status and concise QC comment are recorded.